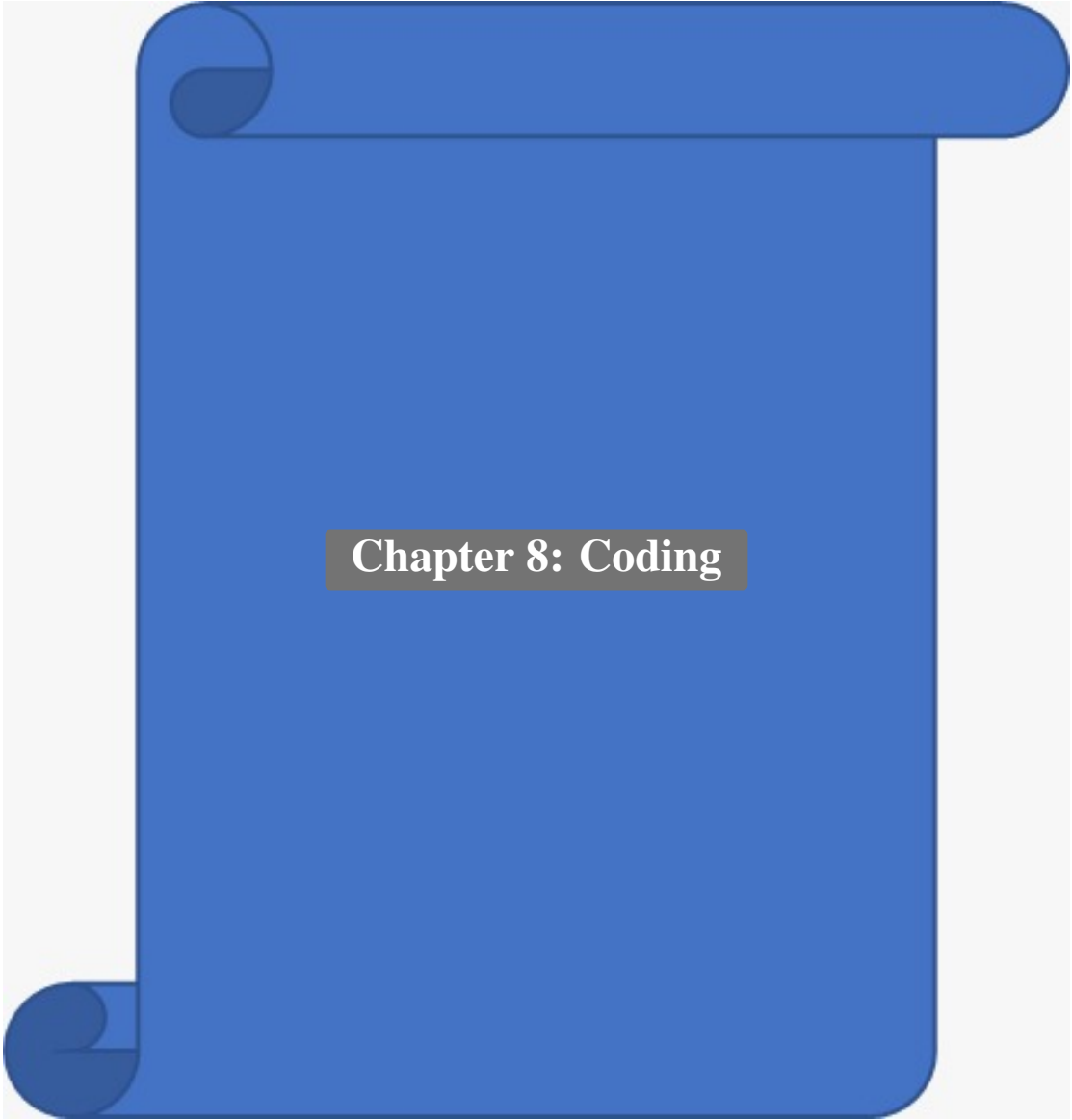




Chapter 8: Coding

This chapter presents the key code modules of the AgriTrace platform. The codebase follows a modular architecture with clear separation of concerns.

8.1 Firebase Configuration

```
1 import { initializeApp, getApps, getApp } from 'firebase/app';
2 import { getAuth } from 'firebase/auth';
3 import { getFirestore } from 'firebase/firestore';
4 import { getStorage } from 'firebase/storage';
5
6 const firebaseConfig = {
7   apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,
8   authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,
9   projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,
10  storageBucket: process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET,
11  messagingSenderId: process.env.
12    NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
13  appId: process.env.NEXT_PUBLIC_FIREBASE_APP_ID,
14 };
15
16 const app = !getApps().length ? initializeApp(firebaseConfig) :
17   getApp();
18 export const auth = getAuth(app);
19 export const db = getFirestore(app);
20 export const storage = getStorage(app);
```

Listing 8.1: Firebase Initialization (firebase.ts)

8.2 Firebase Service Layer

```
1 // Create a new waste listing
2 export async function createListing(
3   data: Omit<Listing, 'id' | 'createdAt' | 'updatedAt' | 'status'
4   '>
5 ): Promise<string> {
6   const docRef = await addDoc(collection(db, 'listings'), {
7     ...data,
8     status: ListingStatus.OPEN,
9     createdAt: serverTimestamp(),
10    updatedAt: serverTimestamp(),
11  });
12  return docRef.id;
```

```

13
14 // Agent claims a listing
15 export async function claimListing(
16   listingId: string, agentId: string, agentEmail: string
17 ): Promise<void> {
18   await updateDoc(doc(db, 'listings', listingId), {
19     status: ListingStatus.ASSIGNED,
20     assignedAgentId: agentId,
21     assignedAgentEmail: agentEmail,
22     updatedAt: serverTimestamp(),
23   });
24 }

```

Listing 8.2: Core Service Functions (firebase-service.ts)

8.3 Authentication Context

```

1 // AuthContext provides authentication state across the app
2 export function AuthProvider({ children }: { children: ReactNode
3   }) {
4   const [user, setUser] = useState<User | null>(null);
5   const [loading, setLoading] = useState(true);
6
7   useEffect(() => {
8     const unsubscribe = onAuthStateChanged(auth, async (
9       firebaseUser) => {
10       if (firebaseUser) {
11         const userDoc = await getDoc(doc(db, 'users',
12           firebaseUser.uid));
13         setUser({ ...firebaseUser, role: userDoc.data()?.role });
14       } else {
15         setUser(null);
16       }
17       setLoading(false);
18     });
19     return () => unsubscribe();
20   }, []);
21
22   return (
23     <AuthContext.Provider value={{ user, loading }}>
24       {children}
25     </AuthContext.Provider>
26   );

```

24 }

Listing 8.3: Auth Context Provider

8.4 Firestore Security Rules

```
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4     function isAuthenticated() {
5       return request.auth != null;
6     }
7
8     match /users/{userId} {
9       allow read: if isAuthenticated();
10      allow create: if isAuthenticated()
11                    && request.auth.uid == userId;
12      allow update: if isAuthenticated();
13      allow delete: if false;
14    }
15
16    match /listings/{listingId} {
17      allow read: if isAuthenticated();
18      allow create: if isAuthenticated();
19      allow update: if isAuthenticated();
20      allow delete: if isAuthenticated()
21                    && request.auth.uid == resource.data.sellerId
22                    ;
23    }
24  }
```

Listing 8.4: Firestore Security Rules (firestore.rules)

8.5 Carbon Credit Calculation

```
1 // Carbon factors (kg CO2 saved per MT of waste diverted)
2 export const CARBON_FACTORS: Record<string, number> = {
3   'Rice Husk': 1320,
4   'Rice Residue': 1280,
5   'Wheat Stubble': 1150,
6   'Sugarcane Bagasse': 980,
7   'Cotton Stalks': 1050,
```

```

8   'Corn Stover': 1100,
9   'Paddy Straw': 1300,
10  'Coconut Shell': 1400,
11  'Groundnut Shell': 1200,
12  'Mustard Stalks': 1080,
13  'Other': 1000,
14 };
15
16 // Calculate carbon offset for diverted waste
17 export function calculateCarbonOffset(
18   wasteType: string, quantityMT: number
19 ): number {
20   const factor = CARBON_FACTORS[wasteType] || CARBON_FACTORS['
    Other'];
21   return factor * quantityMT;
22 }

```

Listing 8.5: Carbon Credit Service (carbon-service.ts)

The `CARBON_FACTORS` constant maps each waste type to its CO₂ equivalent savings in kilograms per metric tonne. The `calculateCarbonOffset` function provides a simple multiplication-based calculation with a fallback to the “Other” category for unrecognized waste types.

8.6 TypeScript Type Definitions

AgriTrace uses TypeScript interfaces and enums to enforce type safety across the entire codebase. The `types.ts` module defines all data models used by the application:

```

1  import { Timestamp } from 'firebase/firestore';
2
3  export enum UserRole {
4    FARMER = 'farmer',
5    AGENT = 'agent',
6    ADMIN = 'admin',
7  }
8
9  export enum WasteReportStatus {
10    PENDING_PICKUP = 'Pending Pickup',
11    IN_TRANSIT = 'In Transit',
12    DELIVERED = 'Delivered',
13    PROCESSING = 'Processing',
14    RECYCLED = 'Recycled',
15    REJECTED = 'Rejected',
16    COMPLETED = 'Completed',

```

```

17 }
18
19 export interface LocationData {
20     latitude: number;
21     longitude: number;
22     address?: string;
23     timestamp?: number;
24 }
25
26 export interface WasteReport {
27     id: string;
28     farmerName: string;
29     farmerId: string;
30     wasteType: string;
31     quantity: number;
32     location: LocationData;
33     photos?: string[];
34     notes?: string;
35     status: WasteReportStatus;
36     collectionAgent?: string;
37     collectionAgentId?: string;
38     reportedAt: Timestamp | Date;
39     lastUpdate: Timestamp | Date;
40     payment?: number;
41     paymentStatus?: 'Pending' | 'Paid';
42 }
43
44 export enum ListingStatus {
45     OPEN = 'OPEN',
46     ASSIGNED = 'ASSIGNED',
47     IN_TRANSIT = 'IN_TRANSIT',
48     DELIVERED = 'DELIVERED',
49     RECYCLED = 'RECYCLED',
50     CANCELLED = 'CANCELLED',
51 }
52
53 export interface Listing {
54     id?: string;
55     title: string;
56     price: number;
57     quantity?: number;
58     location?: LocationData;

```

```

59     category: string;
60     sellerId?: string;
61     sellerEmail?: string;
62     description?: string;
63     photos?: string[];
64     status?: ListingStatus;
65     assignedAgentId?: string;
66     assignedAgentEmail?: string;
67     createdAt?: Timestamp | Date;
68     updatedAt?: Timestamp | Date;
69 }
70
71 export interface UserProfile {
72     uid: string;
73     email: string;
74     role: UserRole;
75     name?: string;
76     phone?: string;
77     location?: LocationData;
78     profilePhoto?: string;
79     createdAt: Timestamp | Date;
80     verified: boolean;
81 }
82
83 export interface CarbonCredit {
84     id?: string;
85     userId: string;
86     userRole: 'farmer' | 'agent' | 'admin';
87     listingId: string;
88     wasteType: string;
89     quantityMT: number;
90     carbonOffsetKg: number;
91     creditDate: Timestamp | Date;
92     status: 'pending' | 'verified' | 'redeemed';
93 }
94
95 export interface Order {
96     id?: string;
97     listingId: string;
98     buyerId: string;
99     sellerId: string;
100    price: number;

```

```

101   status?: 'Pending' | 'Accepted' | 'Rejected' | 'Completed';
102   createdAt?: Timestamp | Date;
103 }

```

Listing 8.6: Type Definitions (types.ts)

The type definitions ensure that all data flowing through the application conforms to a strict schema. The LocationData interface standardizes geolocation data with optional address fields. Enums like ListingStatus and WasteReportStatus enumerate all valid states, preventing invalid status assignments at compile time.

8.7 Location Service

The location service module provides GPS-based geolocation functionality and reverse geocoding:

```

1  export interface Location {
2    latitude: number;
3    longitude: number;
4  }
5
6  export function getCurrentLocation(): Promise<Location> {
7    return new Promise((resolve, reject) => {
8      if (!navigator.geolocation) {
9        reject(new Error('Geolocation is not supported'));
10       return;
11     }
12     navigator.geolocation.getCurrentPosition(
13       (position) => {
14         resolve({
15           latitude: position.coords.latitude,
16           longitude: position.coords.longitude,
17         });
18       },
19       (error) => { reject(error); },
20       { enableHighAccuracy: true, timeout: 15000, maximumAge: 0 }
21     );
22   });
23 }
24
25 export async function reverseGeocode(
26   lat: number, lng: number
27 ): Promise<string> {
28   try {

```



```

29     const response = await fetch(
30         'https://nominatim.openstreetmap.org/reverse' +
31         '?format=json&lat=${lat}&lon=${lng}'
32     );
33     const data = await response.json();
34     return data.display_name || `${lat}, ${lng}`;
35 } catch {
36     return `${lat.toFixed(6)}, ${lng.toFixed(6)}`;
37 }
38 }
39
40 export function calculateDistance(
41     loc1: Location, loc2: Location
42 ): number {
43     const R = 6371; // Earth radius in km
44     const dLat = ((loc2.latitude - loc1.latitude) * Math.PI) / 180;
45     const dLon = ((loc2.longitude - loc1.longitude) * Math.PI) /
46         180;
47     const a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
48         Math.cos((loc1.latitude * Math.PI) / 180) *
49         Math.cos((loc2.latitude * Math.PI) / 180) *
50         Math.sin(dLon / 2) * Math.sin(dLon / 2);
51     const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
52     return R * c;
53 }

```

Listing 8.7: Location Service (location-service.ts)

The location service utilizes the browser's Geolocation API for acquiring GPS coordinates and OpenStreetMap's Nominatim service for reverse geocoding. The `calculateDistance` function implements the Haversine formula to compute the great-circle distance between two geographic points, which is used to sort nearby waste listings for collection agents.

8.8 Notification Service

The notification service manages in-app notifications for status updates and system events:

```

1 import { collection, addDoc, query, where, orderBy,
2     getCountFromServer, updateDoc, doc,
3     serverTimestamp } from 'firebase/firestore';
4 import { db } from '../firebase';
5
6 export interface AppNotification {
7     id?: string;

```

```

8   userId: string;
9   title: string;
10  message: string;
11  type: 'info' | 'success' | 'warning' | 'error';
12  read: boolean;
13  createdAt: any;
14  link?: string;
15 }
16
17 export async function createNotification(
18   data: Omit<AppNotification, 'id' | 'read' | 'createdAt'>
19 ): Promise<string> {
20   const ref = await addDoc(collection(db, 'notifications'), {
21     ...data,
22     read: false,
23     createdAt: serverTimestamp(),
24   });
25   return ref.id;
26 }
27
28 export async function getUnreadCount(
29   userId: string
30 ): Promise<number> {
31   const q = query(
32     collection(db, 'notifications'),
33     where('userId', '==', userId),
34     where('read', '==', false)
35   );
36   const snapshot = await getCountFromServer(q);
37   return snapshot.data().count;
38 }
39
40 export async function markAsRead(
41   notificationId: string
42 ): Promise<void> {
43   await updateDoc(doc(db, 'notifications', notificationId), {
44     read: true,
45   });
46 }

```

Listing 8.8: Notification Service (notification-service.ts)

Notifications are stored as Firestore documents, enabling real-time synchronization. The `getUnreadCount`

function uses Firestore's `getCountFromServer` for efficient counting without downloading entire documents.

8.9 Storage Service

The storage service handles file uploads to Cloudinary for waste report photos and delivery proofs:

```
1  const CLOUD_NAME = process.env.NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME;
2  const UPLOAD_PRESET = process.env.
    NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET;
3
4  export async function uploadImage(
5    file: File
6  ): Promise<string> {
7    const formData = new FormData();
8    formData.append('file', file);
9    formData.append('upload_preset', UPLOAD_PRESET || '');
10   formData.append('folder', 'agritrace');
11
12   const response = await fetch(
13     `https://api.cloudinary.com/v1_1/${CLOUD_NAME}/image/upload`,
14     { method: 'POST', body: formData }
15   );
16
17   if (!response.ok) {
18     throw new Error('Upload failed');
19   }
20   const data = await response.json();
21   return data.secure_url;
22 }
23
24 export async function uploadMultipleImages(
25   files: File[]
26 ): Promise<string[]> {
27   const uploadPromises = files.map((file) => uploadImage(file));
28   return Promise.all(uploadPromises);
29 }
```

Listing 8.9: Storage Service (storage-service.ts)

Image uploads use Cloudinary's unsigned upload preset mechanism, which avoids exposing API secrets on the client side. The `uploadMultipleImages` function leverages `Promise.all` for concurrent uploads, minimizing total upload time when farmers submit multiple waste pho-

tographs.

8.10 Payment Button Component

The PaymentButton component integrates Razorpay's checkout flow with a custom React UI:

```
1 'use client';
2 import { useState } from 'react';
3 import { Button } from './ui/button';
4
5 interface PaymentButtonProps {
6   amount: number;
7   listingId: string;
8   buyerId: string;
9   sellerId: string;
10  onSuccess?: (orderId: string) => void;
11  children?: React.ReactNode;
12  className?: string;
13  variant?: 'default' | 'outline' | 'ghost';
14 }
15
16 export default function PaymentButton({
17   amount, listingId, buyerId, sellerId,
18   onSuccess, children, className, variant = 'default'
19 }: PaymentButtonProps) {
20   const [loading, setLoading] = useState(false);
21
22   const handlePayment = async () => {
23     setLoading(true);
24     try {
25       // Step 1: Create Razorpay order
26       const res = await fetch('/api/razorpay/create', {
27         method: 'POST',
28         headers: { 'Content-Type': 'application/json' },
29         body: JSON.stringify({ amount, listingId, buyerId,
30                               sellerId }),
31       });
32       const { razorpayOrderId } = await res.json();
33
34       // Step 2: Initialize Razorpay Checkout
35       const options = {
36         key: process.env.NEXT_PUBLIC_RAZORPAY_KEY_ID,
```

```

37         currency: 'INR',
38         order_id: razorpayOrderId,
39         handler: async function (response: any) {
40             // Verify payment on server
41             const verifyRes = await fetch('/api/razorpay/
42                 verify', {
43                 method: 'POST',
44                 headers: { 'Content-Type': 'application/
45                     json' },
46                 body: JSON.stringify({
47                     razorpay_order_id: response.
48                         razorpay_order_id,
49                     razorpay_payment_id: response.
50                         razorpay_payment_id,
51                     razorpay_signature: response.
52                         razorpay_signature,
53                     listingId, buyerId, sellerId, price:
54                         amount,
55                 }),
56             });
57             const verifyData = await verifyRes.json();
58             if (verifyData.ok) onSuccess?.(verifyData.
59                 orderId);
60         }
61     };
62     const rzp = new (window as any).Razorpay(options);
63     rzp.open();
64 } catch (err) { console.error('Payment error:', err); }
65 finally { setLoading(false); }
66 };
67
68 return (
69     <Button onClick={handlePayment} disabled={loading}
70         className={className} variant={variant}>
71         {children}
72     </Button>
73 );
74 }

```

Listing 8.10: Payment Button Component (payment-button.tsx)

The payment flow follows a two-step process: (1) a server-side API call creates a Razorpay order with the amount and metadata, and (2) the Razorpay checkout modal is launched on the

client side. This ensures that payment amounts cannot be tampered with by the client.

8.11 Razorpay Payment API Routes

The Next.js API handles server-side order creation and cryptographic verification. The order creation route initiates the transaction with Razorpay:

```
1 import Razorpay from 'razorpay';
2 import { NextResponse } from 'next/server';
3
4 const razorpay = new Razorpay({
5   key_id: process.env.RAZORPAY_KEY_ID!,
6   key_secret: process.env.RAZORPAY_KEY_SECRET!,
7 });
8
9 export async function POST(request: Request) {
10   try {
11     const { amount } = await request.json();
12     const razorpayOrder = await razorpay.orders.create({
13       amount: amount * 100, // paise
14       currency: 'INR',
15       receipt: `receipt_${Date.now()}`,
16     });
17     return NextResponse.json({ razorpayOrderId: razorpayOrder.id });
18   } catch (error) {
19     return NextResponse.json({ error: 'Failed' }, { status: 500 });
20   }
21 }
```

Listing 8.11: Razorpay Order Creation (api/razorpay/create/route.ts)

The function returns the Razorpay order ID which is then used by the client-side checkout modal. After the user completes the payment, the following verification route ensures the payment's cryptographic integrity:

```
1 import { NextResponse } from 'next/server';
2 import crypto from 'crypto';
3
4 export async function POST(req: Request) {
5   try {
6     const { razorpay_order_id, razorpay_payment_id,
7       razorpay_signature, listingId, buyerId,
8       sellerId, price } = await req.json();
```

```

9
10  const body = razorpay_order_id + "|" + razorpay_payment_id;
11  const expectedSignature = crypto
12    .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET!)
13    .update(body.toString())
14    .digest('hex');
15
16  if (expectedSignature !== razorpay_signature) {
17    return NextResponse.json({ error: 'Invalid' }, { status:
18      400 });
19  }
20
21  // Use Firebase Admin for reliable server-side persistence
22  const { db } = await import('@/lib/firebase-server');
23  const { FieldValue } = await import('firebase-admin/firestore
24    ');
25
26  const orderRef = await db.collection('orders').add({
27    listingId, buyerId, sellerId, price,
28    status: 'Paid', createdAt: FieldValue.serverTimestamp(),
29  });
30
31  await db.collection('listings').doc(listingId).update({
32    paymentStatus: 'PAID', orderId: orderRef.id,
33    updatedAt: FieldValue.serverTimestamp(),
34  });
35
36  return NextResponse.json({ ok: true, orderId: orderRef.id });
37 } catch (err) {
38   return NextResponse.json({ error: 'Error' }, { status: 500 })
    ;
  }
}

```

Listing 8.12: Razorpay Payment Verification (api/razorpay/verify/route.ts)

This verification step is critical for security as it prevents malicious users from forging payment success responses. By performing this check on the server using the RAZORPAY_KEY_SECRET and then updating the database using the Firebase Admin SDK, we ensure that the transaction is legitimate and the listing status is accurately reflected.

8.12 Admin Statistics Service

The admin statistics function aggregates platform-wide metrics by querying multiple Firestore collections:

```
1 export async function getAdminStats() {
2   try {
3     const usersRef = collection(db, 'users');
4     const farmersQuery = query(usersRef,
5       where('role', '==', 'FARMER'));
6     const agentsQuery = query(usersRef,
7       where('role', '==', 'AGENT'));
8
9     const [farmersSnap, agentsSnap] = await Promise.all([
10       getCountFromServer(farmersQuery),
11       getCountFromServer(agentsQuery),
12     ]);
13
14     const listingsRef = collection(db, 'listings');
15     const openQuery = query(listingsRef,
16       where('status', '==', 'OPEN'));
17     const deliveredQuery = query(listingsRef,
18       where('status', '==', 'DELIVERED'));
19
20     const [openSnap, deliveredSnap] = await Promise.all([
21       getCountFromServer(openQuery),
22       getCountFromServer(deliveredQuery),
23     ]);
24
25     return {
26       users: {
27         farmers: farmersSnap.data().count,
28         agents: agentsSnap.data().count,
29         total: farmersSnap.data().count +
30           agentsSnap.data().count,
31       },
32       listings: {
33         open: openSnap.data().count,
34         delivered: deliveredSnap.data().count,
35       },
36     };
37   } catch (error) {
38     return {
```



```

39     users: { farmers: 0, agents: 0, total: 0 },
40     listings: { open: 0, delivered: 0 },
41   };
42 }
43 }

```

Listing 8.13: Admin Stats (firebase-service.ts – getAdminStats)

The function uses `getCountFromServer` for efficient aggregation without downloading individual documents, reducing bandwidth consumption. Error handling returns zeroed-out stats to prevent the admin dashboard from crashing if Firestore is temporarily unavailable.

8.13 Waste Delivery Tracking

The waste delivery tracking functions manage the logistics of waste transportation from collection agents to recycling facilities:

```

1  export async function recordWasteDelivery(
2    data: any
3  ): Promise<string> {
4    const payload = {
5      ...data,
6      createdAt: serverTimestamp(),
7      status: 'DELIVERED',
8    };
9    const ref = await addDoc(
10      collection(db, 'wasteDeliveries'), payload
11    );
12
13    if (data.reportId) {
14      const reportRef = doc(db, 'wasteReports', data.reportId);
15      await updateDoc(reportRef, {
16        status: 'IN_TRANSIT',
17        collectionAgentId: data.agentId,
18        collectionAgent: data.agentName,
19        lastUpdate: serverTimestamp(),
20      });
21    }
22    return ref.id;
23  }
24
25  export async function recordWasteProcessing(
26    data: any
27  ): Promise<string> {

```

```

28  const payload = {
29      ...data,
30      createdAt: serverTimestamp(),
31      status: 'RECYCLED',
32  };
33  const ref = await addDoc(
34      collection(db, 'wasteProcessing'), payload
35  );
36
37  if (data.intakeId) {
38      const intakeRef = doc(db, 'wasteIntakes', data.intakeId);
39      await updateDoc(intakeRef, {
40          status: 'PROCESSED',
41          lastUpdate: serverTimestamp(),
42      });
43  }
44  return ref.id;
45  }

```

Listing 8.14: Waste Delivery Functions (firebase-service.ts)

These functions implement a cascading update pattern where recording a delivery or processing event automatically updates the parent document's status. This ensures data consistency across the waste management pipeline without requiring client-side orchestration.

8.14 Listing Status Workflow Functions

The following functions implement the complete listing status transition workflow:

```

1  export async function startPickup(
2      listingId: string
3  ): Promise<void> {
4      const ref = doc(db, 'listings', listingId);
5      await updateDoc(ref, {
6          status: 'IN_TRANSIT' as ListingStatus,
7          updatedAt: serverTimestamp(),
8      });
9  }
10
11 export async function markDelivered(
12     listingId: string
13 ): Promise<void> {
14     const ref = doc(db, 'listings', listingId);
15     await updateDoc(ref, {

```

```

16     status: 'DELIVERED' as ListingStatus,
17     updatedAt: serverTimestamp(),
18   });
19 }
20
21 export async function cancelListing(
22   listingId: string
23 ): Promise<void> {
24   const ref = doc(db, 'listings', listingId);
25   await updateDoc(ref, {
26     status: 'CANCELLED' as ListingStatus,
27     updatedAt: serverTimestamp(),
28   });
29 }

```

Listing 8.15: Listing Workflow (firebase-service.ts)

Each function targets a specific listing document by ID and atomically updates both the status field and the updatedAt timestamp. The status transitions follow a strict sequence: OPEN → ASSIGNED → IN_TRANSIT → DELIVERED → RECYCLED, with CANCELLED available from any active state.